UNIVERSITY OF BOTSWANA

CSI473 Project Design Report

TRANS-UB - University of Botswana Student Marketplace Hub

Team 9 | Phase 2 Project Design Report

Field	Complete
Assignment/phase	Phase 2 Project Design Report (Assignment 3)
Approved project title	TRANS-UB - University of Botswana Student Marketplace Hub
Approval decision/date	Approved with minor conditions; approval evidence carried forward from Phase 1
Team number	9
Members and IDs	Letlhabile Xoliso Serufo - 202302072 Lebogang Rantlheng - 202400279 Tshegofatso Botho Wakwena - 202302811 Amanda Mapete - 202303699 Lame Vambu - 202203427
Repository URL	https://github.com/lebogang00279/Trans-UB
Submission tag	phase2-submission
Date	30 October 2026

1. Executive summary

TRANS-UB is a student-to-student marketplace for the University of Botswana. Phase 2 carries the approved Phase 1 design into an implementable vertical slice centred on UC-05, Reserve an Available Listing. The implemented slice is intentionally narrow: it demonstrates the reservation workflow end to end rather than claiming that every marketplace function has been implemented.

The runnable implementation uses Python and SQLite and exposes POST /api/listings/{listingId}/reserve. It validates the buyer and listing, creates a Pending Order, records the initial OrderStatusChange, changes the Listing from Available to Reserved in the same core transaction, and attempts seller notification only after the reservation state has committed.

Seven automated tests were executed successfully with zero failures. The tests cover the happy path, invalid input, the own-listing rule, ineligible buyers, duplicate prevention, simultaneous reservation attempts and notification-failure tolerance. A real HTTP smoke test also returned a Pending Order with the Listing in Reserved state.

The principal Phase 2 release conclusion is therefore conditional: the UC-05 academic vertical slice is runnable and internally consistent, but TRANS-UB remains a prototype. Student verification is synthetic, notification delivery is simplified, automatic expiry is not scheduled, and the wider FR-01 to FR-10 marketplace is not fully implemented.

2. Problem evidence, stakeholders and scope

2.1 Problem evidence carried forward

Phase 1 evidence established that UB student trading occurs through fragmented channels such as student messaging groups, Facebook posts, notice boards and word of mouth. The approved design therefore focuses on structured approved listings, reservation state, moderation and transaction-linked reputation.

Evidence	What it established
Student buy-and-sell group screenshots	Active trading exists but listings and seller history are difficult to retain and compare.
Buyer/seller interviews	Trust, visibility and reliable reservation status are recurring concerns.
Public sales posts	Comparable items are advertised separately, making structured comparison difficult.

2.2 Stakeholders

Stakeholder	Phase 2 relevance
Student Buyer	Primary actor for the implemented UC-05 reservation slice.
Student Seller	Owns the listing and receives a reservation notification.
Platform Administrator	Design responsibility retained for moderation; not part of the runnable UC-05 slice.
SRC	External escalation participant for serious disputes; not part of the runnable slice.

2.3 Scope and constraints

- Low-risk physical goods remain the approved product scope.
- Food, medicines, alcohol, stolen/restricted goods and high-risk services remain excluded.
- Payment and physical handover remain outside TRANS-UB under D-001.
- Student verification remains synthetic/prototype-only and does not represent official UB integration.
- Phase 2 implementation evidence is limited to the approved UC-05 reservation vertical slice.

3. Glossary

Term	Definition
Available	Approved Listing that can currently be reserved.

Reserved	Listing with one live reservation/order.
Pending	Order created by a successful reservation and awaiting seller response.
OrderStatusChange	Persistent audit record of an Order state transition.
Live order	For the implemented integrity constraint: Pending, Accepted or Delivered.
NotificationPort	Boundary through which the core workflow requests seller notification.
Vertical slice	A runnable end-to-end path through interface/API, application logic and persistence.
Synthetic identity	Prototype-only student account data, not official UB records.

4. Functional requirements and use cases

ID	Requirement	Phase 2 status
FR-01	Prototype account/login with simulated student verification.	Design baseline; not implemented in UC-05 slice.
FR-02	Seller creates/edits listing.	Design baseline.
FR-03	Administrator approves/rejects/removes listing.	Design baseline; seed data supplies approved listing.
FR-04	Buyer searches/browses/filters/compares listings.	Design baseline.
FR-05	Buyer reserves/orders an Available listing.	Implemented and tested.
FR-06	Seller accepts/rejects Pending order.	Design/state model retained.
FR-07	Expiry/cancellation releases listing.	Design/state model retained.
FR-08	Delivered/Completed lifecycle; Listing becomes Sold.	Design/state model retained.
FR-09	One review only after Completed order.	Design baseline.
FR-10	Report/moderate/escalate disputes.	Design baseline.

TRANS-UB Use Case Model

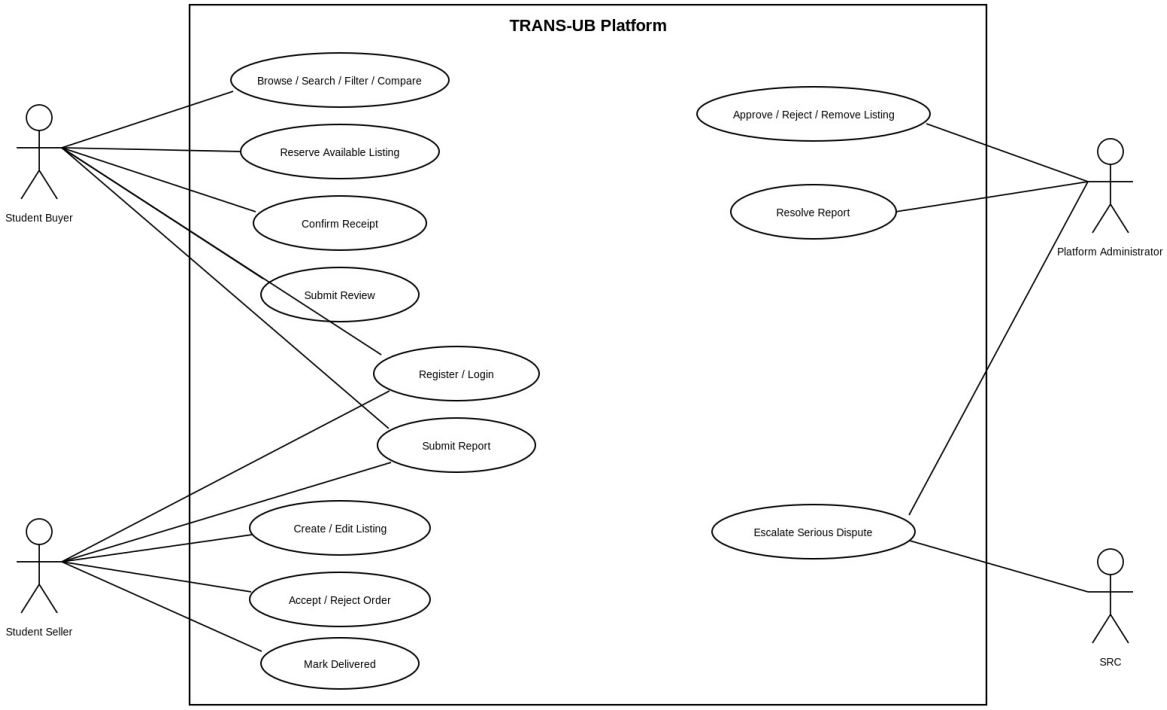


Figure 1. TRANS-UB requirements-aligned use-case model.

4.1 Implemented detailed use case: UC-05

1. Buyer selects Reserve on an Available listing.
2. System validates buyer identity/eligibility and rejects own-listing reservation.
3. System performs a final availability check.
4. System creates a Pending Order and initial OrderStatusChange.
5. System changes Listing Available -> Reserved atomically with the Order creation.
6. After commit, seller notification is attempted.
7. The API returns the Order reference, Pending status, Reserved listing status and response deadline.

5. Quality requirements

Quality concern	Design obligation	Implemented evidence
Duplicate prevention	At most one live reservation per Listing.	Partial unique index + concurrent/duplicate tests.
Transaction consistency	Order creation and Listing reservation cannot diverge.	SQLite BEGIN IMMEDIATE transaction and rollback behavior.
Stale availability	Final availability check before commit.	Service checks status and guarded UPDATE.
Failure tolerance	Notification failure must not roll back reservation.	Notification executed after commit; failure test passes.
Traceability	Requirement -> design -> code -> test -> result.	Reconciliation matrix in Phase 2 evidence.

6. Analysis/domain model and responsibilities

The Phase 2 implementation preserves the Phase 1 domain vocabulary. The runnable slice persists StudentAccount, Listing, Order, OrderStatusChange and Notification information. The wider Review, Report and administrator responsibilities remain design-level artifacts.

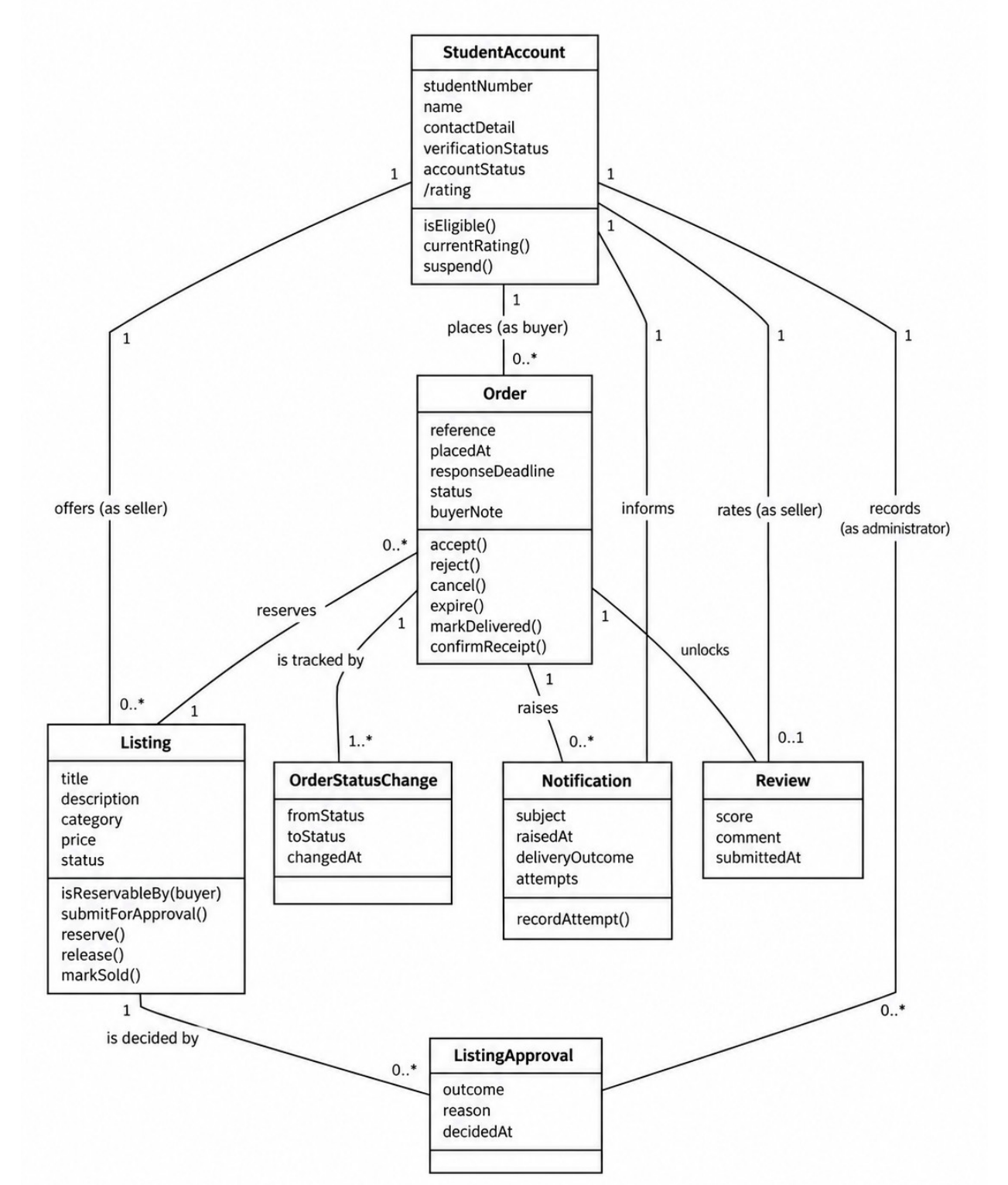


Figure 2. Phase 1 domain model carried into Phase 2 design.

Responsibility	Owner
Buyer eligibility	StudentAccount / reservation validation
Listing availability and ownership	Listing
Order lifecycle data	Order
State-history evidence	OrderStatusChange
UC-05 coordination and transaction boundary	ReservationService
Notification delivery boundary	NotificationPort

7. Interaction and lifecycle

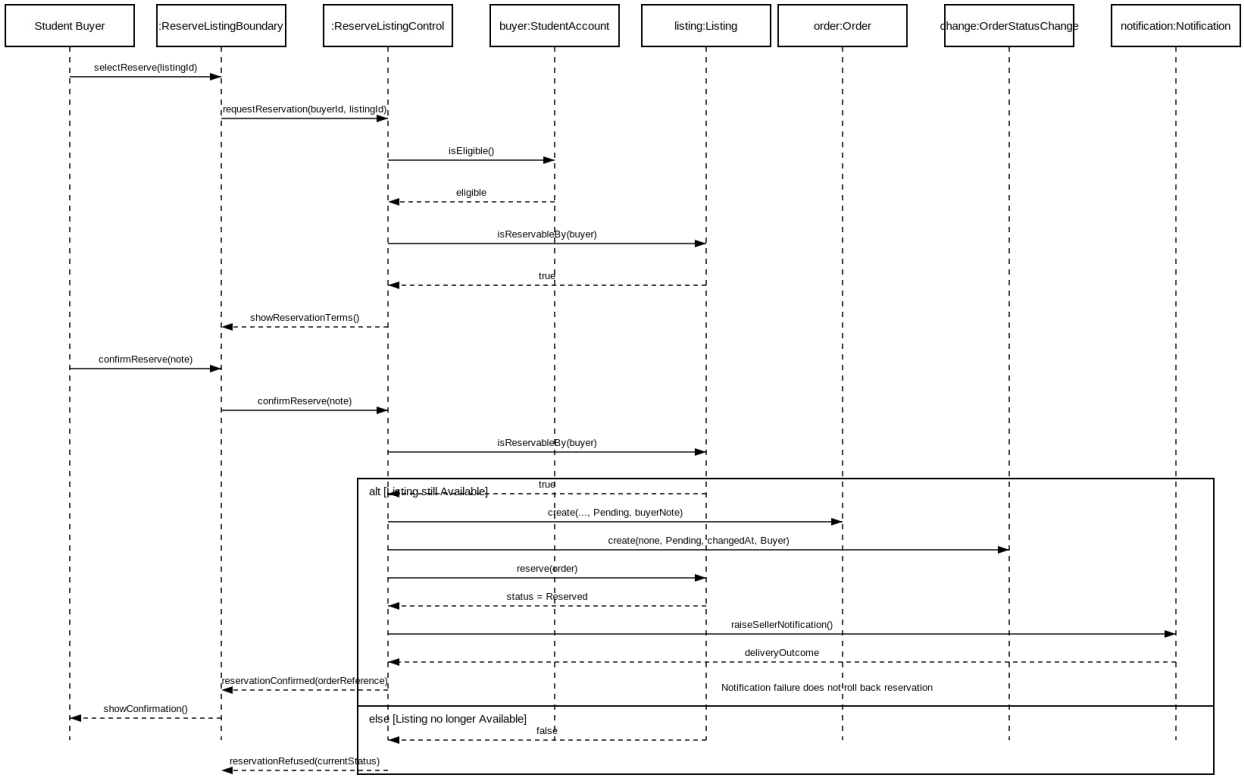


Figure 3. UC-05 reservation interaction model.

The implementation follows the sequence-model intent: validation and the final listing check precede Order creation; reservation state is committed before notification; and the failure of notification is isolated from the core transaction.

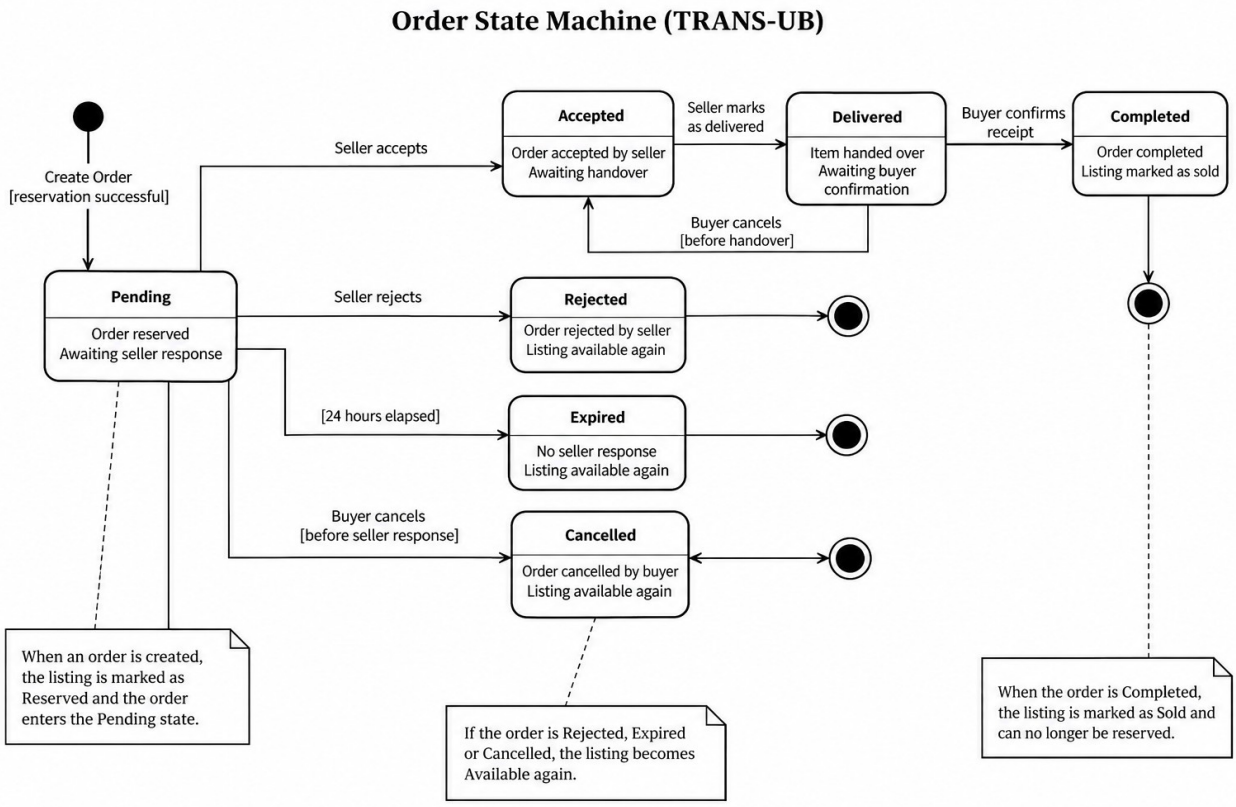


Figure 4. Order state machine retained as the full lifecycle design.

Only the Pending creation transition is exercised by the Phase 2 runnable slice. Seller acceptance/rejection, expiry, cancellation, delivery and completion remain part of the approved lifecycle design but are not claimed as implemented in this vertical slice.

8. Architecture and decisions

8.1 Selected architecture

TRANS-UB retains the modular layered monolith selected in Phase 1/Lab 7. The Phase 2 slice implements the same direction in a compact form: HTTP boundary -> ReservationService -> persistence transaction, with notification isolated as a secondary dependency.

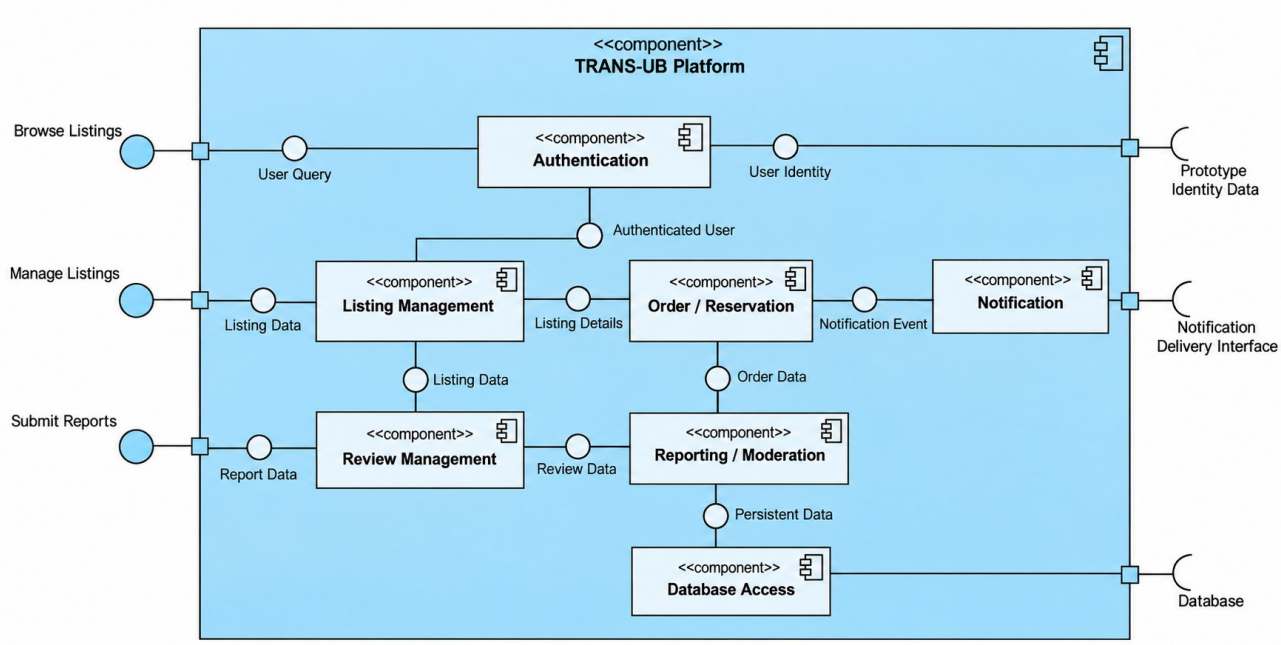


Figure 5. TRANS-UB component architecture.

Decision	Outcome
D-001	No online payment or platform-controlled physical handover.
ADR-001	Use modular layered monolith rather than distributed services.
ADR-002	Core workflow depends on stable persistence/notification ports rather than concrete external technology.

8.2 Consequences

- Single-process deployment keeps the prototype simple and allows one transaction boundary.
- Clear module/port boundaries improve testability and reduce infrastructure coupling.
- The prototype remains unsuitable for production without stronger identity, security, operational monitoring and deployment hardening.

9. Data, API/service and deployment design

9.1 Logical data model

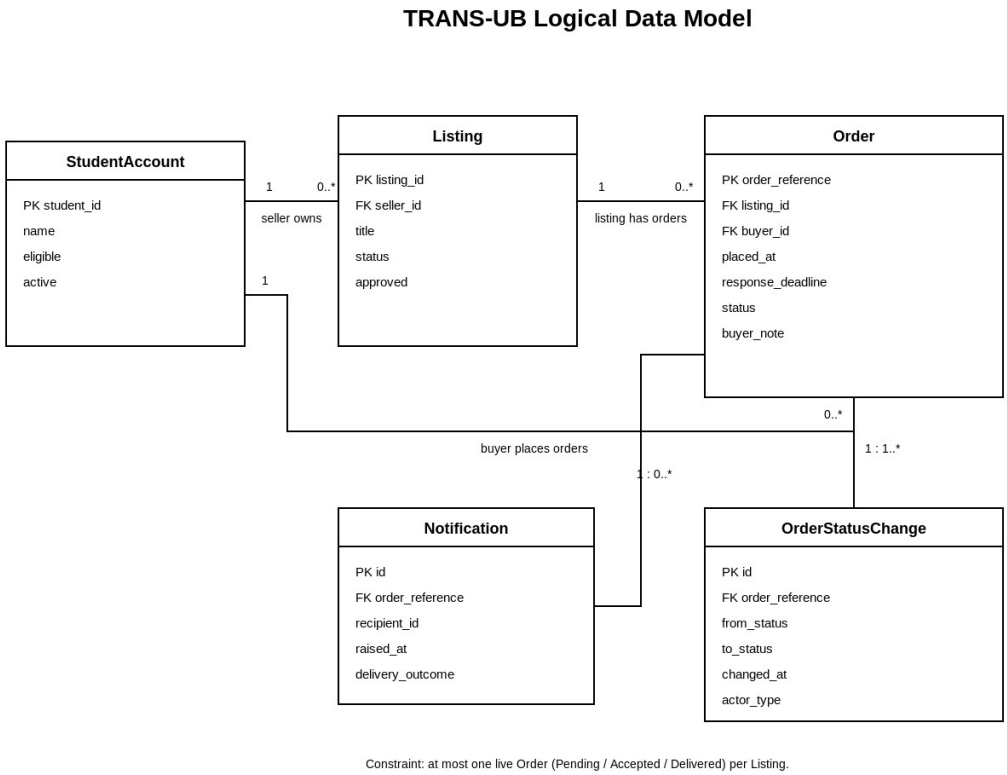


Figure 6. Logical data model implemented by schema/schema.sql.

The database schema enforces referential relationships and defines a partial unique index on orders(listing_id) for live states Pending, Accepted and Delivered. This complements the application-level availability check and is the strongest persistence-level protection against duplicate live reservations.

9.2 Core API/service contract

`POST /api/listings/{listingId}/reserve`

Input / output	Contract
Path	listingId identifies the Listing to reserve.
Request body	buyerId required; buyerNote optional.
201 success	Pending Order, Reserved Listing, responseDeadline and notificationOutcome returned.
400	INVALID_REQUEST
403	BUYER_NOT_ELIGIBLE or OWN_LISTING
404	BUYER_NOT_FOUND or LISTING_NOT_FOUND
409	LISTING_NOT_AVAILABLE or DUPLICATE_RESERVATION

9.3 Deployment

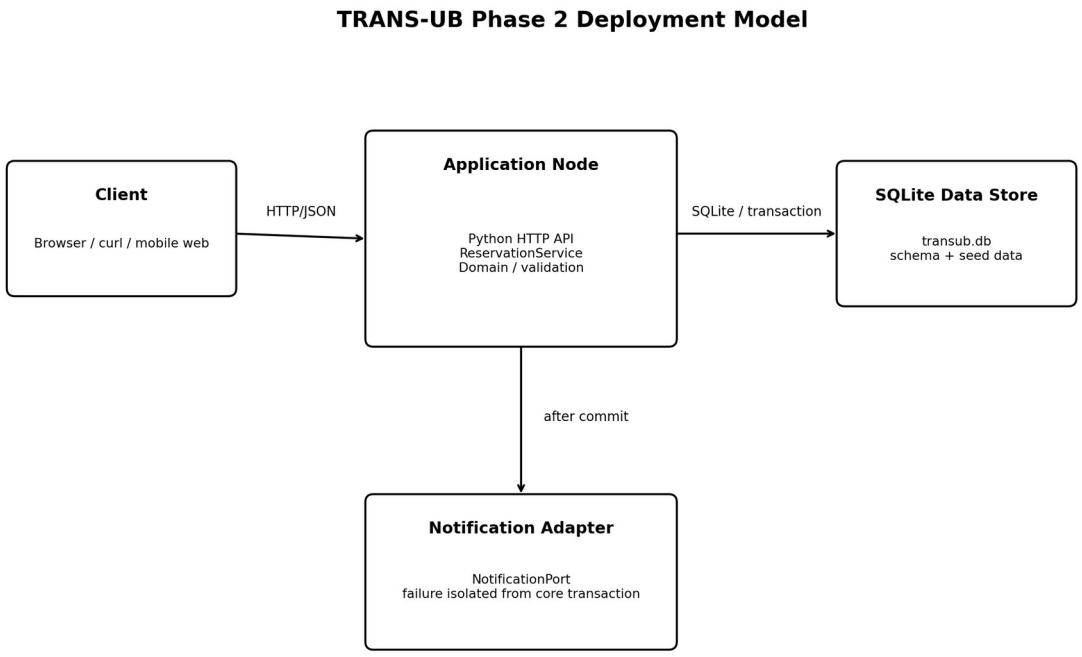


Figure 7. Phase 2 runtime/deployment structure.

9.4 Failure/recovery behavior

The core reservation state is committed before seller notification. If notification raises an exception, the service records outcome FAILED but leaves the Order Pending and Listing Reserved. If reservation validation or persistence fails before commit, the transaction rolls back and the service returns an error without partial reservation state.

10. Detailed design, patterns and framework

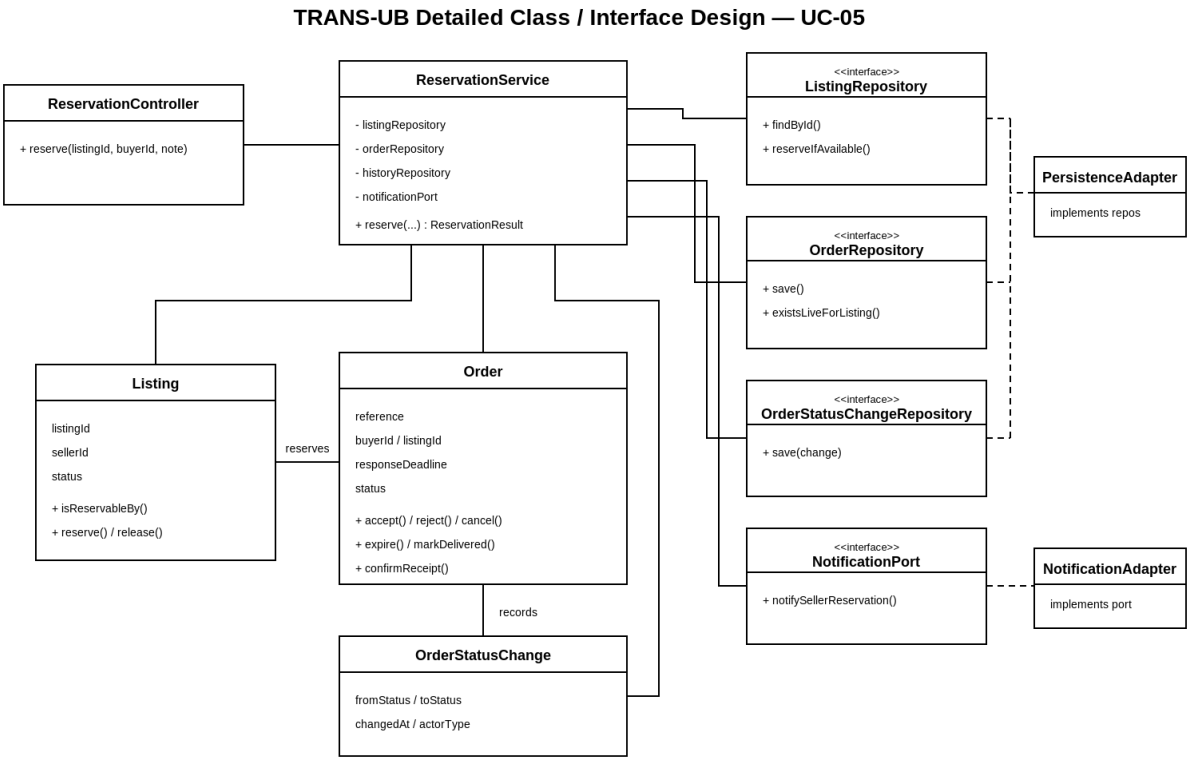


Figure 8. Detailed class/interface design for the UC-05 workflow.

Problem	Selected technique	Reason
Persistence coupling	Repository/port boundary	Keeps core workflow independent of database-specific code.
Workflow coordination	Application Service / ReservationService	Creates one explicit entry point and transaction boundary.
Notification dependency	Dependency inversion / NotificationPort	Allows failure isolation and provider substitution.
Order states	Full State pattern rejected	Current lifecycle can be expressed more simply through controlled transitions.

The runnable slice is intentionally framework-light: Python standard-library HTTP handling and sqlite3 are used so the vertical slice remains self-contained. This is a Phase 2 implementation choice for demonstrability, not a claim that these libraries are the final production technology stack.

11. User-interface, accessibility and usability

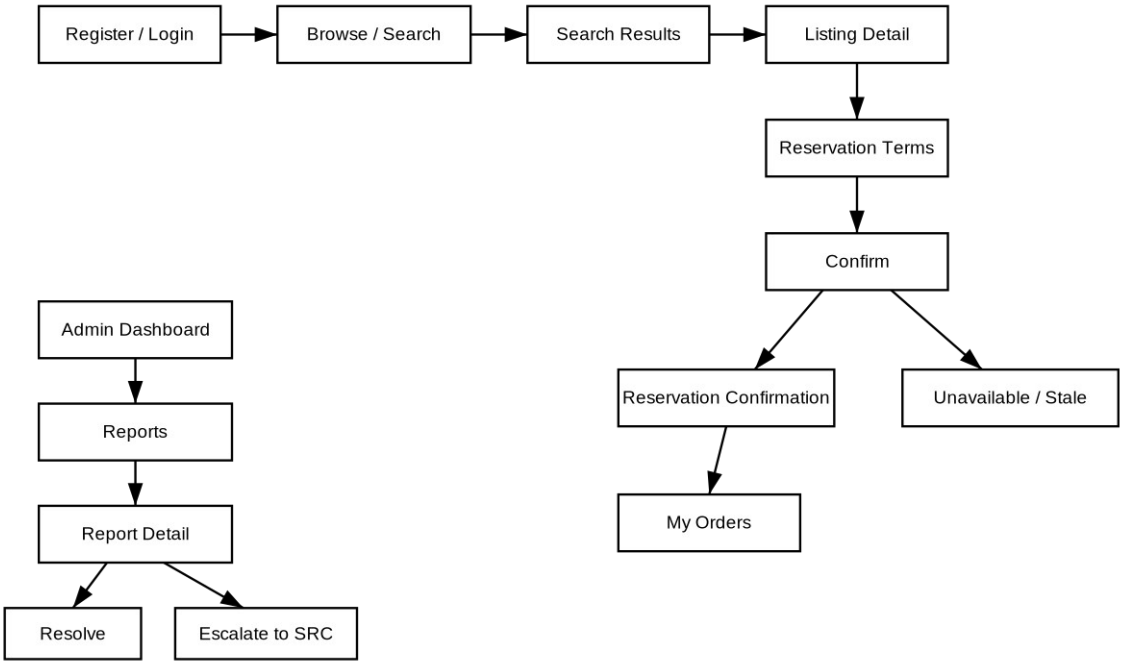


Figure 9. Navigation model for buyer reservation and administrator reporting tasks.

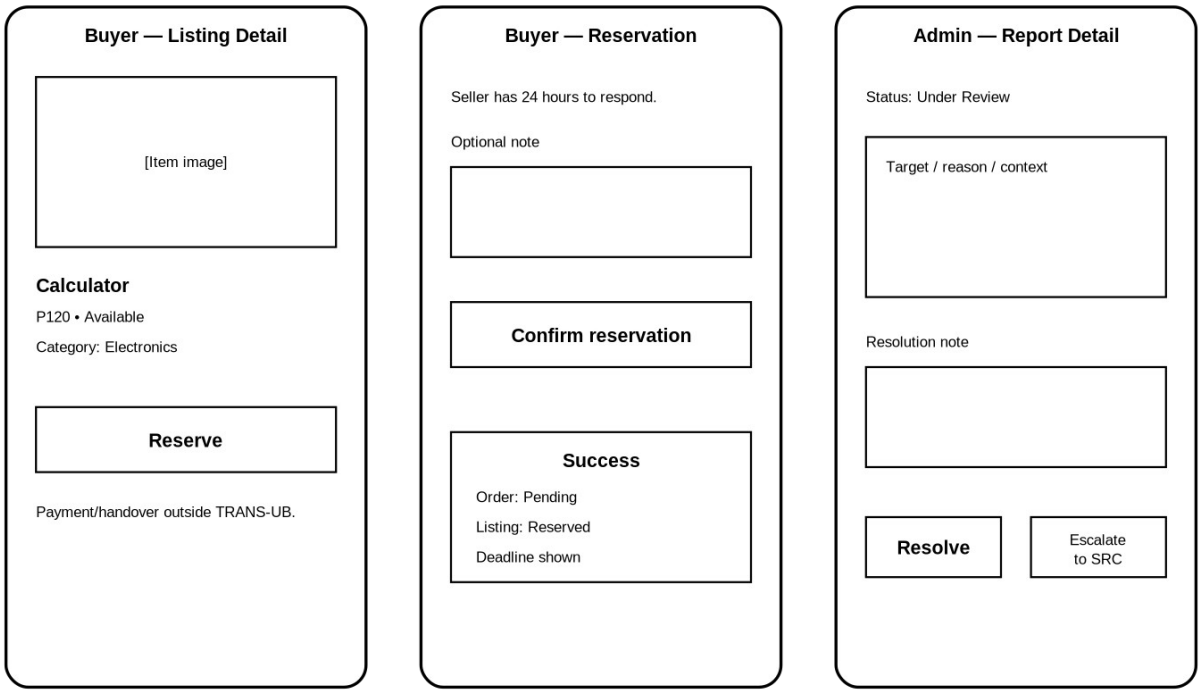


Figure 10. Low-fidelity task-focused wireframes from Lab 10.

The Phase 2 runnable slice exposes the reservation operation through HTTP rather than a finished graphical UI. Therefore the Lab 10 navigation, wireframes and accessibility criteria are retained as implementation obligations rather than falsely reported as completed UI behavior.

- Status must be communicated using text, not colour alone.
- The reservation screen must disclose the 24-hour response rule.
- A stale/unavailable result must explain that no new Order was created.
- An unknown/offline outcome should direct the buyer to check existing orders before retrying.

The three-person usability record should contain real participant observations before it is used as portfolio evidence; no participant results are fabricated in this report.

12. Implementation vertical slice

12.1 Repository structure

```
src/transub/
  api.py
  db.py
  service.py
  setup_db.py
tests/
  test_reservation.py
schema/
  schema.sql
Docs/
Evidence/
README.md
```

12.2 Setup and run

```
python3 -m src.transub.setup_db
python3 -m src.transub.api
python3 -m unittest discover -s tests -v
```

12.3 Implemented transaction

8. Begin an immediate SQLite transaction.
9. Validate buyer existence, active/eligible state and own-listing restriction.
10. Load Listing and require approved + Available.

11. Insert the Pending Order.
12. Insert OrderStatusChange from none to Pending.
13. Guardedly update Listing from Available to Reserved.
14. Commit the core transaction.
15. Attempt notification after commit and persist DELIVERED or FAILED outcome.

13. Testing and results

The automated test suite was executed again during final report preparation. Seven tests passed, with zero failures and zero errors.

Test	Expected behavior	Result
test_success	Pending Order created; Listing Reserved; status history recorded.	PASS
test_invalid	Missing buyer input rejected.	PASS
test_own_listing	Buyer cannot reserve own Listing.	PASS
test_ineligible	Ineligible buyer rejected.	PASS
test_duplicate	Second live reservation prevented.	PASS
test_concurrent_exactly_one	Two concurrent attempts produce exactly one success.	PASS
test_notification_failure	Notification failure does not roll back reservation.	PASS

```
Ran 7 tests in 0.149s

OK
```

13.1 HTTP smoke test

```
POST /api/listings/LIST-104/reserve

Observed:
status = Pending
listingStatus = Reserved
notificationOutcome = DELIVERED
```

The generated order reference and timestamps vary on each run; therefore they are not treated as fixed expected values.

14. Reconciliation and change history

Requirement / concern	Design evidence	Code evidence	Test/result
FR-05 / UC-05 reservation	Sequence + API contract	src/transub/service.py	test_success — PASS
Duplicate prevention	Quality/architecture constraint	schema unique live-order index + transaction	test_duplicate — PASS
Simultaneous requests	Architecture risk	BEGIN IMMEDIATE + guarded update	test_concurrent_exactly_one — PASS
Own-listing rule	API validation	ReservationService	test_own_listing — PASS
Buyer eligibility	API validation	ReservationService	test_ineligible — PASS
Notification failure tolerance	Quality scenario / NotificationPort	notification after commit	test_notification_failure — PASS

14.1 Honest differences from the broader design

- Only UC-05 is implemented end to end; the remaining functional requirements remain design-level evidence.
- The implementation uses a compact ReservationService and SQLite rather than a complete production framework stack.
- The database schema includes only entities required by the reservation vertical slice.
- The current HTTP vertical slice has no finished graphical UI.
- Automatic 24-hour expiry is represented by responseDeadline data but no background scheduler is implemented.

14.2 Change history

Phase / lab	Change
Phase 1	Reconciled direct reservation workflow, lifecycle terminology and traceability.
Lab 7	Selected modular layered monolith and documented architecture drivers.
Lab 8	Defined logical data, reservation API and failure-integrity obligations.
Lab 9	Defined application-service and dependency-boundary decisions.
Lab 10	Prepared navigation, wireframes, UI states and accessibility obligations.
Lab 11 / Phase 2	Implemented UC-05 runnable slice, schema and automated tests; recorded reconciliation and residual risk.

15. Risks, technical debt and release recommendation

ID	Residual issue	Priority / consequence
TD-01	Synthetic student verification only.	Medium — academic prototype cannot claim official UB identity assurance.
TD-02	Notification delivery uses simplified prototype behavior.	Medium — retry/backoff/provider integration remain future work.
TD-03	Only UC-05 is implemented.	High for full-product release; acceptable for approved vertical-slice demonstration.
TD-04	No automatic 24-hour expiry scheduler.	Medium — deadline is stored but automation is incomplete.
TD-05	No production security/deployment hardening.	High for real deployment.

Release recommendation: CONDITIONALLY SUITABLE FOR ACADEMIC PHASE 2 DEMONSTRATION OF UC-05. The implementation and tests support the reservation vertical slice, but the system should not be described as production-ready or as a complete implementation of all TRANS-UB requirements.

16. Academic integrity and AI-use record

Generative AI assistance was used to help structure documentation, create and revise repository evidence, produce diagrams, implement a small runnable UC-05 vertical slice, and execute/review automated tests in a controlled environment. The generated implementation and report were checked against the approved TRANS-UB requirements and the actual test output. The team remains responsible for verifying, understanding and presenting every submitted artefact, and for disclosing assistance according to University of Botswana course requirements.

No simulated test result is reported as an executed result in this report. The seven PASS results in Section 13 are based on an actual execution of the packaged unittest suite. Usability observations are not fabricated.

References

16. University of Botswana. CSI473 Project Design Report Template, Semester 1, 2026/27.
17. TRANS-UB GitHub repository: https://github.com/lebogang00279/Trans-UB
18. TRANS-UB Phase 1 Project Design Report and phase1-submission baseline.
19. TRANS-UB D-001 - Offline Payment and Physical Handover.
20. TRANS-UB architecture options, quality-to-architecture mapping and ADR-001.
21. TRANS-UB core reservation API contract and logical data/deployment design.
22. TRANS-UB pattern decisions, framework decision and ADR-002.
23. TRANS-UB Phase 2 runnable source, schema, automated tests and test-results summary.

Appendices

Appendix A - Automated test execution record

```
test_concurrent_exactly_one ... ok
test_duplicate ... ok
test_ineligible ... ok
test_invalid ... ok
test_notification_failure ... ok
test_own_listing ... ok
test_success ... ok

Ran 7 tests in 0.149s
OK
```

Appendix B - Core endpoint example

```
POST /api/listings/LIST-104/reserve
Content-Type: application/json

{
  "buyerId": "202302072",
  "buyerNote": "Campus collection please"
}
```

Appendix C - Phase 2 submission evidence

- Runnable source under src/.
- Automated tests under tests/.
- Database schema under schema/schema.sql.
- Reconciliation matrix and technical-debt/risk register.
- Final Phase 2 report PDF under submissions/.
- Final repository tag: phase2-submission.